Autonomous Lane Following & Obstacle Avoidance

Bosch Future Mobility Challenge (BFMC) 2026

ROS Noetic · NVIDIA Jetson Orin · Intel RealSense · Gazebo · YOLOv8

Andre Tiu | *University of Nevada, Las Vegas / Street Precision Las Vegas*

github.com/Free-Triall/BFMC-Street-Precision-Las-Vegas

About the Competition

The **Bosch Future Mobility Challenge (BFMC)** is an annual international autonomous driving competition organised by Bosch and hosted in Cluj-Napoca, Romania. Student engineering teams from universities around the world are given a standardised 1:10 scale vehicle and tasked with programming it to navigate a structured urban environment entirely autonomously — no remote control, no human intervention during the run.

The track simulates real-world urban driving conditions. Teams must handle lane keeping at speed, recognise and respond to traffic signs, negotiate intersections, and safely react to dynamic obstacles such as pedestrians and other vehicles. Everything must run in real time on embedded hardware mounted on the car itself.

> **Why it is challenging:** Unlike many robotics competitions, BFMC combines *perception*, *planning*, and *control* into a single integrated system that must perform reliably on a physical track under competition conditions. There is no ground truth feed — the car must determine where it is, what surrounds it, and what to do entirely from its own sensors.

In 2026, our team represented *[Your University]* among dozens of international competitors. While we did not advance to the semi-finals, the competition served as a rigorous, real-world engineering challenge that pushed the team to build a complete autonomous driving stack from the ground up — from writing the first line of code to running the car on the competition track in Romania.

Contributions

This was a five person team with responsibilities split across software, hardware, and mechanical domains:

- **Andre & Dany** — jointly designed and implemented the vision-based lane following system, including the full image processing pipeline, the Stanley steering controller, motion-based obstacle detection, and the lateral avoidance manoeuvre logic. We also integrated the NVIDIA Jetson Orin and Intel RealSense camera with the vehicle's PCB, and built the ground-truth simulation arbiter used during Gazebo testing.
- **Vanshdeep & Kyle** — developed and tuned a YOLOv8-based object detection model for classifying traffic signs and obstacles from the camera feed.
- **Ashlynn** — handled mechanical engineering responsibilities including lubricating and tuning the drivetrain, adjusting the gear system for reliable performance, and designing and producing custom 3D printed parts for the vehicle.

This write-up focuses on the lane following, obstacle avoidance, and hardware integration work, which were my primary technical contributions to the project.

Hardware Platform

Component	Device	Role
Compute	NVIDIA Jetson Orin	ROS host, all inference & control
Primary camera	Intel RealSense (gesture/depth)	Lane vision, obstacle detection
Object detection	YOLOv8 (teammate)	Traffic sign & obstacle classification
Vehicle platform	BFMC 1:10 scale car	Actuation via `/automobile/command`
OS / Middleware	Ubuntu 20.04 + ROS Noetic	Node communication (topics, timers)

Simulation Environment

Before any code ran on the physical car, the entire stack was developed and validated inside a **Gazebo** simulation of the BFMC track. Gazebo provides a physics-based 3D environment where the virtual car receives real camera images and responds to the same ROS commands as the real vehicle — meaning code written for simulation transfers directly to hardware with minimal changes.

Working in simulation first gave the team a fast iteration loop — a steering tuning change could be tested in seconds rather than requiring the physical car to be set up each time. It also allowed us to stress-test edge cases such as losing lane markings mid-corner or encountering a stationary obstacle blocking the path, without any risk of damaging the hardware.

System Architecture

The software stack is organised as a pipeline of ROS nodes, each with a single clear responsibility:

1. **Lane Follower** (`lane_follow_bfmc`) — reads the raw camera stream, detects lane markings, and publishes a combined speed and steering command to the `/cmd/lane` topic.
2. **Obstacle Arbiter** — sits between the lane follower and the car's actuators. It listens to `/cmd/lane` and decides whether to pass the command through or override it with a full stop. Two versions exist: a vision-based arbiter embedded in the lane follower node, and a ground-truth Gazebo arbiter (`gt_obstacle_arbiter`) used during simulation.
3. **YOLO Detection Node** (teammate) — runs YOLOv8 on the camera feed in parallel to classify traffic signs and detected objects, publishing results as separate ROS topics.
4. **Vehicle Interface** — the BFMC firmware receives JSON-formatted commands on `/automobile/command` and drives the steering servo and motor accordingly.

Lane Following

How It Works

Lane following is the core behaviour of the vehicle. The goal is to keep the car centred between the lane markings at all times — through curves, at speed, and with the imperfect images produced by a real camera on a moving vehicle.

My approach processes each camera frame through the following stages:

1. **Region-of-interest crop** — only the lower portion of the frame is analysed. The sky and distant scenery are discarded as they contain no useful lane information.
2. **Brightness thresholding** — a pixel brightness cutoff isolates white lane markings from the dark track surface. Morphological operations (opening and closing) then remove noise and fill small gaps in the detected lines.
3. **Sliding window search** — the image is divided into nine horizontal strips. Starting from the bottom, a search window finds clusters of white pixels in each strip, tracking the left and right lane boundaries upward through the frame.
4. **Polynomial curve fitting** — a second-order polynomial is fitted through the detected pixels on each side, giving a smooth mathematical description of where each lane boundary lies, even around bends.
5. **Centre and heading estimation** — two points along the fitted curves (one near, one further ahead) compute two key values: the *cross-track error* (how far left or right of centre the car is) and the *heading error* (the angle between the car's current direction and the lane direction).
6. **Stanley steering control** — a control law from autonomous vehicle research combines heading error and cross-track error into a single steering angle command. An additional gain multiplier activates on sharp corners where stronger correction is needed.
7. **Speed scheduling** — the car automatically slows down in tight turns and during obstacle avoidance, then speeds back up on straights.

Key Tuned Parameters

Parameter	Value	Effect
Cross-track gain (`k_stanley`)	7.5	How aggressively to correct lateral drift
Heading gain (`k_heading`)	3.8	How aggressively to correct angle error
Corner boost (`turn_boost_gain`)	10.65	Extra steering authority on sharp curves
Steering smoothing	0.20	Reduces jitter in the output steering signal
Speed range	0.08–0.16 m/s	Slower in turns, faster on straights

Obstacle Detection & Avoidance

Detecting Obstacles Without Depth Data

Rather than relying solely on depth measurements, I implemented a motion-based detection approach using the standard camera. The key insight is that stationary background pixels remain consistent between frames, while an approaching obstacle causes pixels in the forward view to change. The pipeline:

1. Compare consecutive camera frames pixel by pixel within a defined front-centre region of interest.
2. Threshold the difference image to isolate significant changes, then group changed pixels into contiguous regions.
3. Classify the largest detected region by its horizontal position — *left*, *right*, or *center* of the lane.

4. Require the obstacle to appear consistently across multiple consecutive frames before triggering any response, to avoid reacting to noise or sudden lighting changes.

How the Car Responds

The response depends on both the size of the detected region (a proxy for how close the obstacle is) and its lateral position in the frame:

Situation	Position	Response
Very large (very close)	Anywhere	Full stop until obstacle moves
Large and blocking	Centre	Full stop until obstacle moves
Moderate size	Left	Shift trajectory rightward
Moderate size	Right	Shift trajectory leftward
Obstacle clears	—	Resume normal driving

The avoidance manoeuvre works by shifting the target lane centre laterally in the vision pipeline — rather than commanding a raw steering angle override, the system instructs the lane follower to treat a shifted centre as the new target. This causes the existing Stanley controller to steer around the obstacle naturally, keeping the avoidance behaviour smooth and consistent with normal driving.

Simulation Arbiter (Ground-Truth Mode)

During Gazebo testing a separate node, `gt_obstacle_arbiter`, was used in place of camera-based detection. It reads the exact positions of all objects in the simulation directly from Gazebo and computes their distance and lateral offset from the ego vehicle using pose transforms. This enabled deterministic obstacle testing — verifying stop and resume logic was correct before trusting it to the vision system.

Results & Reflections

What We Achieved

The lane following system performed consistently in both simulation and on the physical competition track, maintaining lane centre through straights and moderate curves. The motion-based obstacle detector reliably triggered stops and avoidance manoeuvres in simulation and in controlled physical tests. The modular ROS architecture meant each component could be tested and debugged independently, which significantly reduced development time.

What We Would Do Differently

Competing in BFMC 2026 taught the team a great deal about the gap between simulation and real hardware. Key lessons learned:

- **The steering actuator dead-zone was a significant challenge.** The physical servo required a minimum input of around 7.22° before it moved at all, causing the car to drift on gentle curves. This required careful physical tuning that could not be replicated in simulation.
- **Motion-based obstacle detection has limits.** It works well for moving obstacles but is sensitive to vehicle vibration on rough track sections. Incorporating the RealSense depth stream for metric distance estimates would have been considerably more robust.
- **More physical track time is essential.** Many parameters were tuned in simulation and adjusted on the day. Structured hardware testing sessions earlier in the project cycle would

have allowed proper data-driven tuning before the competition.

Future Directions

- Replace the threshold-based lane detector with a lightweight CNN trained on BFMC-specific track data for better generalisation to varying lighting conditions.
- Integrate RealSense depth data for metric obstacle distance, removing reliance on pixel-area thresholds.
- Build a proper finite-state machine to handle intersections, traffic lights, and sign-based rules as a unified high-level behaviour layer.
- Migrate the full stack to ROS 2 for improved real-time performance and long-term maintainability.

Code & Resources

Full source code is available on GitHub. The repository contains the lane follower node, both arbiter implementations, ROS launch files, and parameter configuration notes.

github.com/Free-Triall/BFMC-Street-Precision-Las-Vegas

Technologies used: Python 3 · ROS Noetic · OpenCV · NumPy · Gazebo · YOLOv8 · NVIDIA Jetson Orin · Intel RealSense

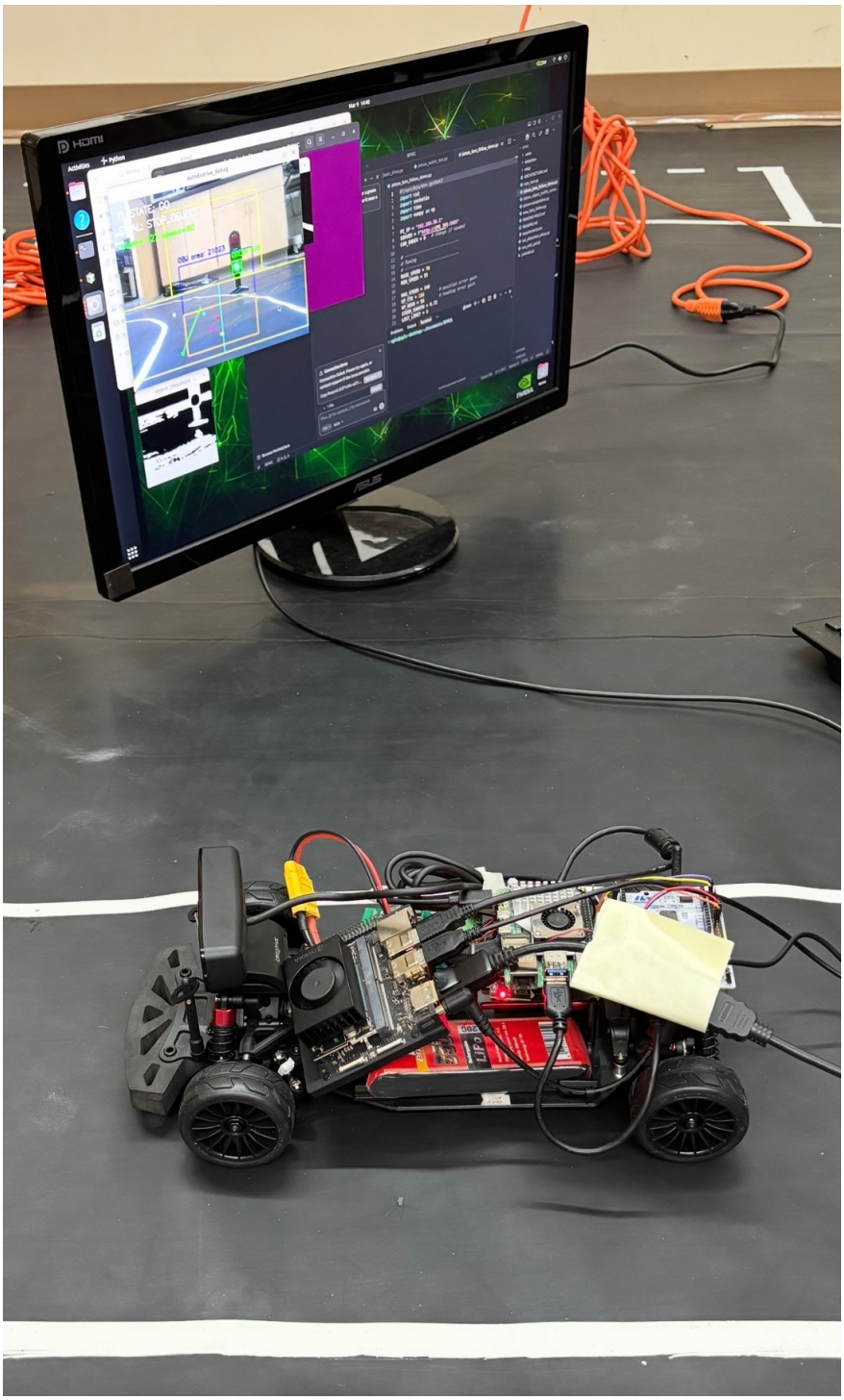


Figure 1: Our competition vehicle equipped with the NVIDIA Jetson Orin and Intel RealSense camera.

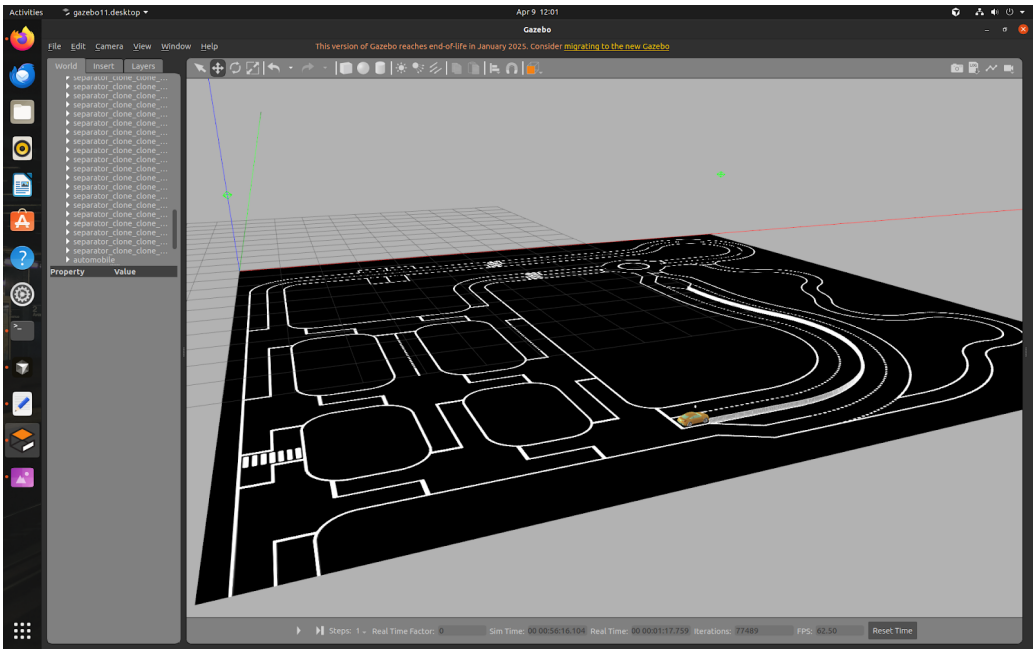
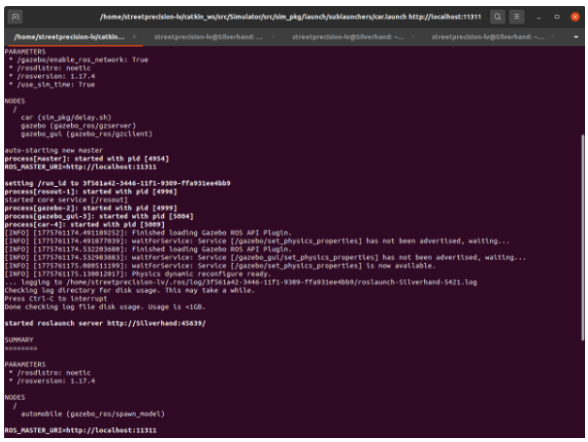
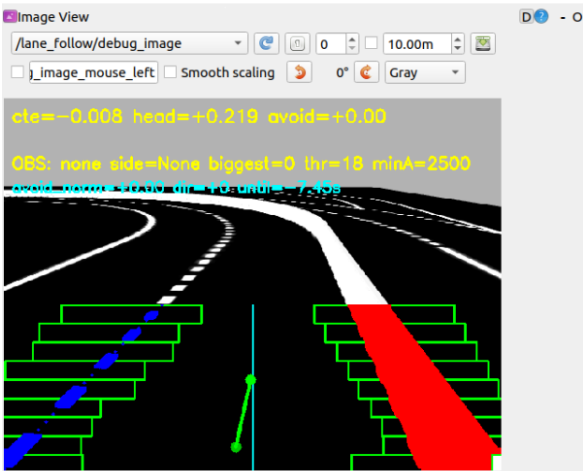


Figure 2: The Gazebo simulation environment replicating the BFMC urban track, used for development and testing prior to hardware deployment.



(a) ROS Noetic and Gazebo launch confirmation.



(b) RQT image view with live obstacle bounding boxes.

Figure 3: Simulation validation screenshots confirming the full node graph and camera pipeline before hardware testing.